Exam 4 Review

Greatest hits of the semester

Sam Ogden

DRAFT

This slide deck is still under active development. Expect changes before it is finalized.

How to study this deck

- Use the weekly study guides as a starting point.
- Focus on the **mechanism**, the state it manages, and the tradeoffs it creates.
- Be ready to compare different **policies** and explain why one exists instead of another.
- If you can explain the idea in your own words, you are usually in good shape.

Exam overview

- 20-25 questions, mostly fill-in-the-blank, calculation, and short answer
- 110 minutes
- handwritten
- closed book, no internet
- one handwritten note sheet, both sides
- bring a dedicated calculator; phones do not count
- roughly 25% new material

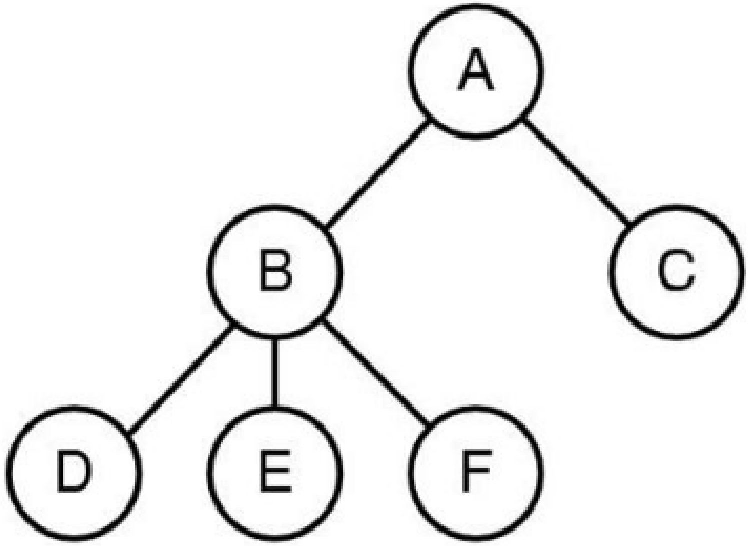
How questions are graded

- `8-point` questions are usually calculation or tracing problems.
- `4-point` questions check core ideas from lecture.
- `2-point` and `1-point` questions check whether you can connect ideas or reason from first principles.
- The point value is not a guide to how much work the question takes; the lower-point questions are often the more lateral or subtle ones.
- I do want the **correct term** when you know it, but a clear explanation of the idea can still earn credit.
- The goal is to understand the **mechanism**, not just the buzzword.

Processes and Scheduling

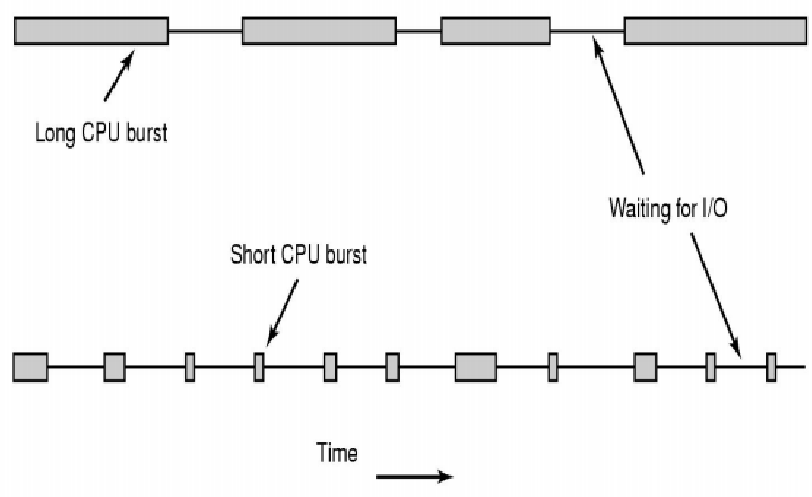
What runs, when it runs, and why the scheduler makes the choice

Processes



- A program is a file on disk; a process is the running instance.
- `fork()` clones the current proces `exec()` replaces its code, and `wait()` reaps children.
- The OS manages processes, and the shell is another process that launches them.
- On questions, draw the process tree and the ready/running/blocked/terminat state diagram first.
- Then reason about what state

Scheduling

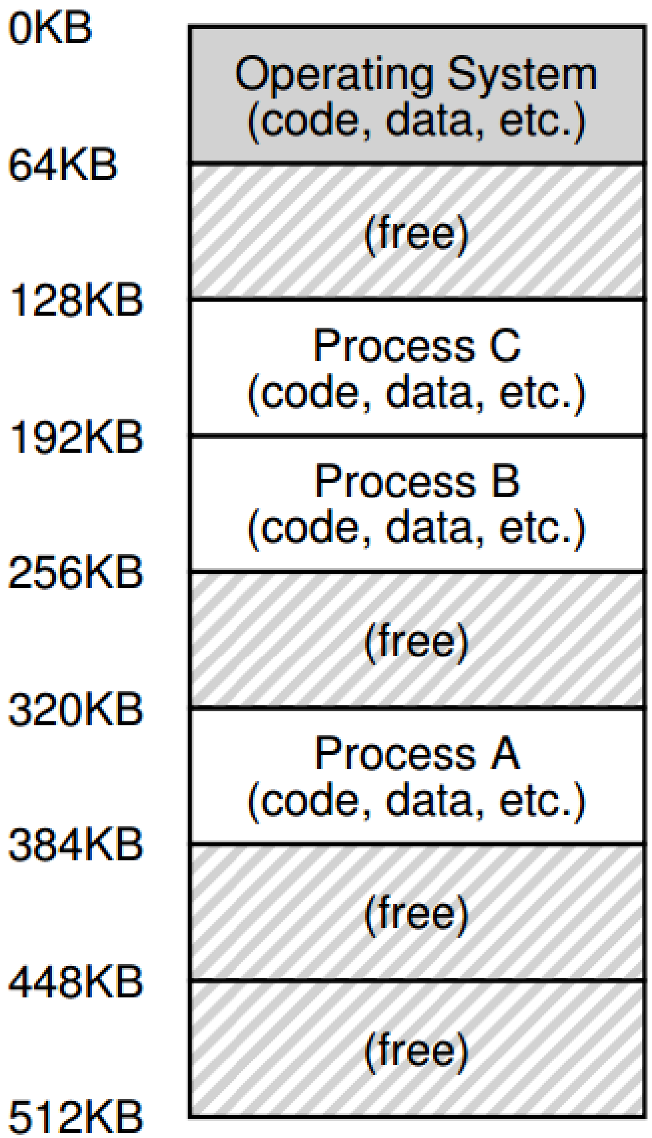


- Know the scheduling metrics: turnaround, response, utilization, and fairness.
- FCFS, SJF/STCF, and RR each optimize different goals and fail in different ways.
- MLFQ tries to favor short or interactive jobs while still giving CPU-bound jobs time.
- Be ready to simulate the next scheduling decision and explain the tradeoff being made.

Virtual Memory

How the machine gives each process its own illusion of memory

Address translation



- Think of translation as something that happens when code calls `read_byte(vaddr)` or `write_byte(vaddr, value)`.
- Virtual memory gives each process the illusion that it has its own private memory.
- Base-and-bounds, segmentation, paging, TLBs, and swapping are different **mechanisms** for implementing that illusion.

Memory questions to be able to answer

- Translate a simple address by hand.
- Say what happens on a TLB miss.
- Explain why page tables can be hierarchical.
- Describe what swapping buys you and what it costs.
- Explain how free-space structures and fragmentation show up in memory management.

Concurrency

Why shared state makes everything interesting

Threads and locks

```c
#include <stdio.h>
#include <assert.h>
#include <pthread.h>

void *mythread(void *arg) {
    printf("%s\n", (char *) arg);
    return NULL;
}

int
main(int argc, char *argv[]) {
    pthread_t p1, p2;
    int rc;
    printf("main: begin\n");
    rc = pthread_create(&p1, NULL, mythread, "A"); assert(rc == 0);
    rc = pthread_create(&p2, NULL, mythread, "B"); assert(rc == 0);
    // join waits for the threads to finish
    rc = pthread_join(p1, NULL); assert(rc == 0);
    rc = pthread_join(p2, NULL); assert(rc == 0);
    printf("main: end\n");
    return 0;
}
```

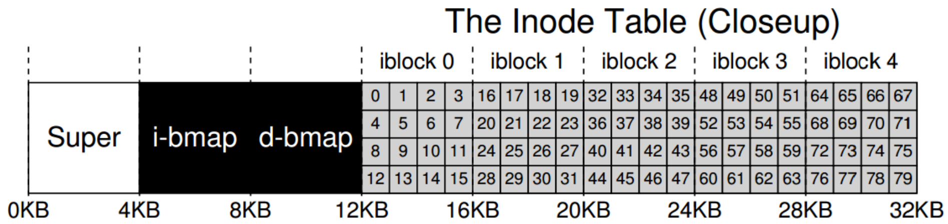
- Threads share an address space; processes mostly do not.
- A critical section is any code that must not interleave with conflicting access.
- Locks protect invariants; races happen when the wrong interleaving matters.
- Know when a bug needs a lock versus a different

Condition variables and semaphores

Storage and Filesystems

How the machine turns bytes into names, metadata, and blocks

Devices, disks, and files



- Devices are driven by controllers, drivers, interrupts, DMA, and sometimes polling I/O.
- HDDs care about seek time, rotational delay, and transfer time.
- Files, directories, links, and mount points are just the visible surface.
- Underneath, a file system is a giant data structure built from inodes.

File-system implementation

What operation happens between these two states?

```
inode bitmap  1110
inodes        [d a:0 r:3] [d a:1 r:3] [d a:2 r:2] []
data bitmap   1110
data          [(.,0) (..,0) (u,1)] [(.,1) (..,0) (k,2)] [(.,2) (..,1)] []
```

Command: [Select]

```
inode bitmap  1111
inodes        [d a:0 r:3] [d a:1 r:3] [d a:2 r:3] [d a:3 r:2]
data bitmap   1111
data          [(.,0) (..,0) (u,1)] [(.,1) (..,0) (k,2)] [(.,2) (..,1) (s,3)]
[(.,3) (..,2)]
```

- Be able to reason about what changes when a file is created, removed, linked, or written.
- Metadata updates matter just as much as the data itself.
- Be ready to trace inode and block-pointer changes.
- If you can trace the before/after state.

Languages

How source text becomes something the machine can run

Grammars, lexers, and parsers

```
expr ::= expr + term | term
term ::= term * factor | factor
factor ::= ( expr ) | id | num
```

- BNF describes structure; the lexer turns characters into tokens.
- Predictive parsing uses grammar structure and lookahead to choose productions.
- Know the difference between **terminals**, **non-terminals**, and **productions**.
- If you can sketch an unambiguous parse tree, you parse the grammar.

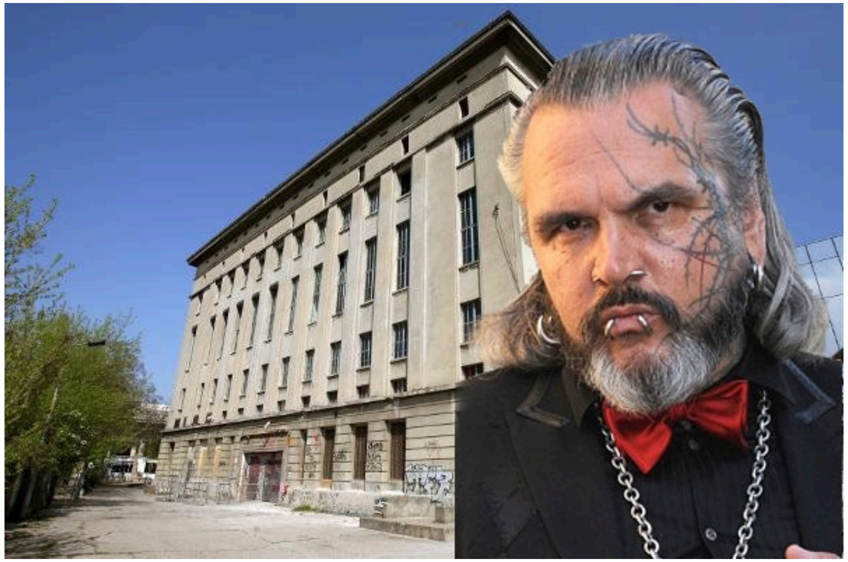
Compilers and interpreters

- A compiler **emits** code or another lower-level representation.
- An interpreter **evaluates** the program directly.
- The front end usually goes: source text -> tokens -> parse tree / AST -> emit or eval.
- Know where `emit()` and `eval()` happen and what each one receives.

Security

Who are you, what can you do, and how do we keep that safe?

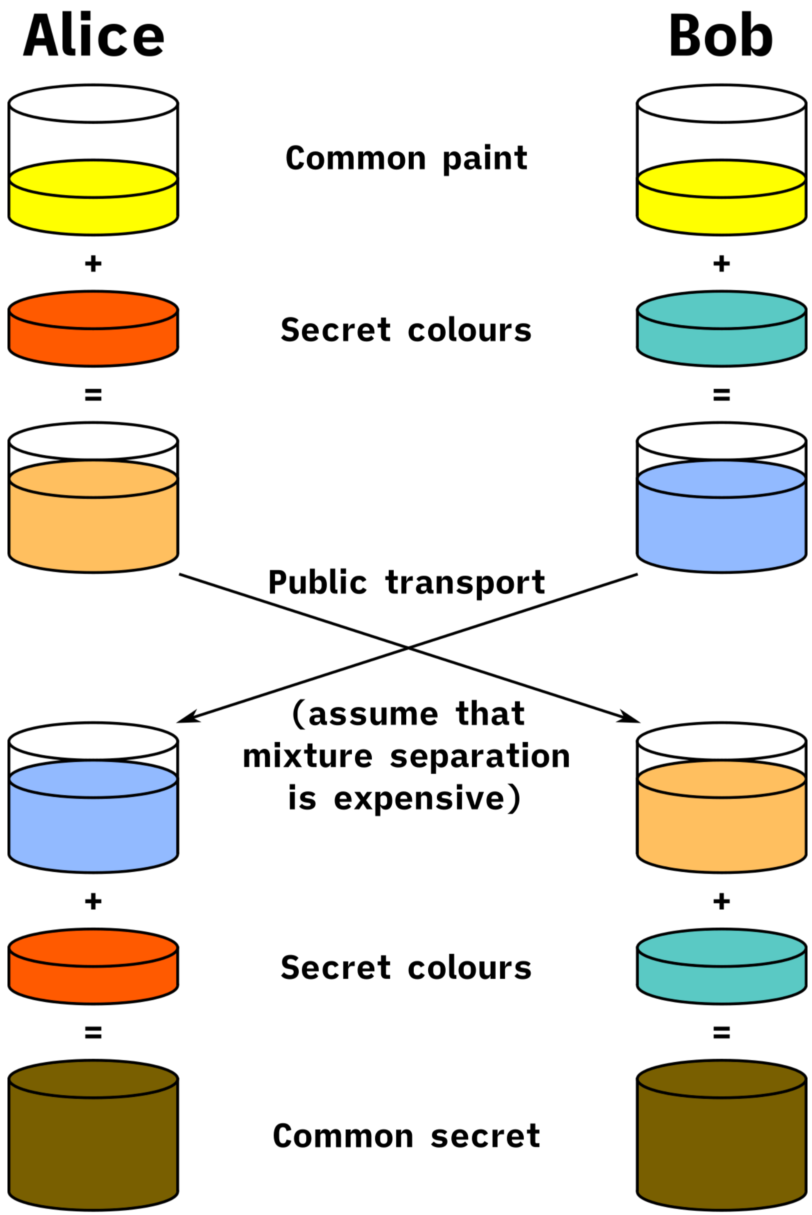
Authentication and access control



- **Authentication** answers “who are you?”
- **Authorization** answers “are you allowed?”
- Know the common **authentication** factors, salts, and why password hashes need help.
- Password hashing is about slowing attackers down, not making secrets magically safe.
- ACLs, capabilities, RBAC, and

Access control

Cryptography



- **Hashes, signatures, symmetric crypto**, and **public-key crypto** solve different problems.
- **Key exchange** is about getting a shared secret safely; trust is the hard part.
- Common failure modes are using the wrong primitive or using a primitive incorrectly.
- Be able to say what problem a crypto tool is actually solving.

Final checklist

- Can you explain each **mechanism** in your own words?
- Can you compare two **policies** and say what each one costs?
- Can you identify what state the OS or runtime is protecting?
- Can you say why the “obvious” answer is not always the right one?